

Introduction of DBMS (Database Management System)

What is Database?

Data refers to raw, unorganized facts and figures, such as numbers, text, images, or symbols, that can be processed and analyzed to extract meaningful information

- Data can exist in a raw form (unorganized) or processed form (organized and meaningful).
- A database is a structured collection of data designed for efficient storage, retrieval and manipulation.
- It serves as a centralized repository, allowing data to be accessed, managed, and updated by multiple users or application

A high-performing database is vital for any organization, supporting operations, customer interactions and systems like digital libraries, reservations, and inventory management. Databases are essential because they:

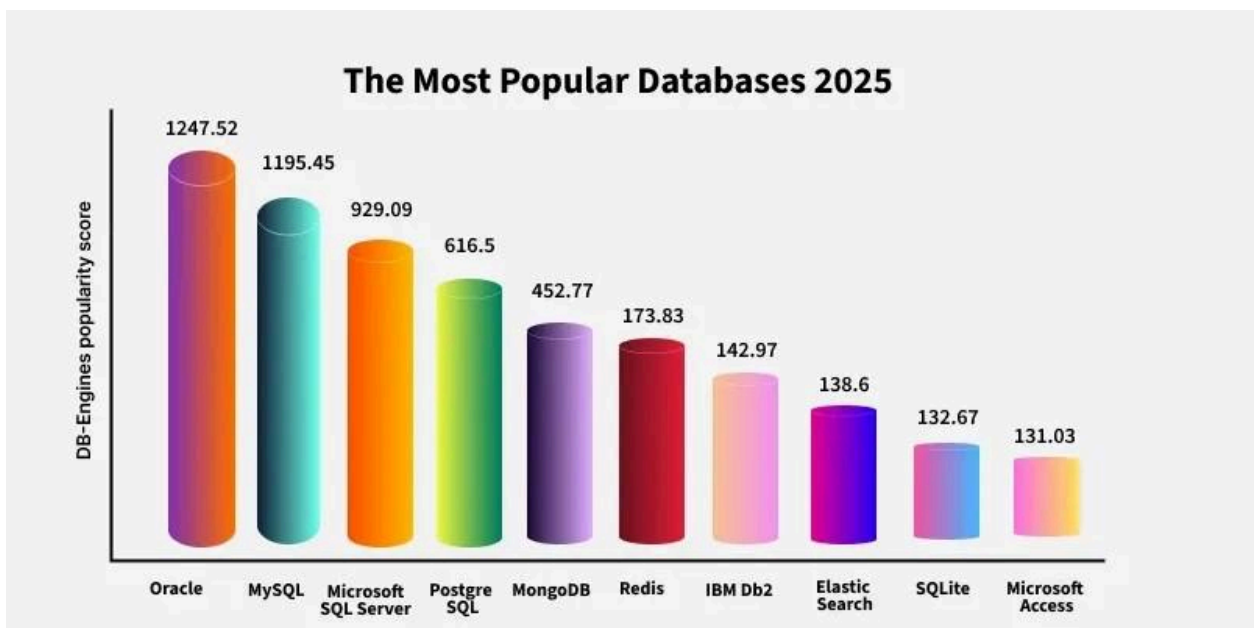
- **Scale efficiently** to handle massive volumes of data.
- **Ensure data integrity** through built-in rules and constraints.
- **Protect data** with secure access controls and compliance support.
- **Enable analytics** by identifying trends and guiding informed business decisions.

Working of Databases

Databases work by organizing and storing information in a structured or unstructured format, allowing easy access, retrieval, and modification. At the core of every database system is the **Database Management System (DBMS)**—a software layer that acts as an intermediary between users and the raw data.

- The DBMS handles tasks like querying, updating, deleting and managing access permissions, without requiring users to know the physical details of where data is stored.
- When a user submits a request (such as a search or update), the DBMS processes the query, locates the relevant data, and returns results in a structured format.
- DBMSs provide features like backup, recovery, performance optimization and data security to ensure the system runs efficiently and reliably.

According to industry rankings, the most popular databases of 2025 are:



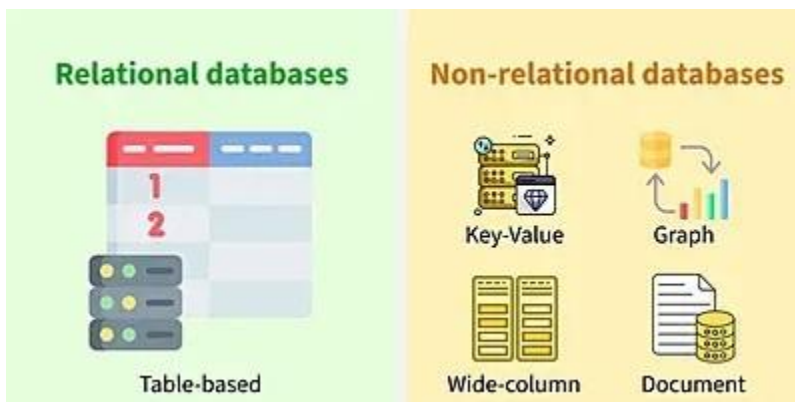
Components of a Database

Databases consist of several critical components that work together to **store, organize** and **retrieve data** effectively. Here is a detailed explanation of each component:

- **Data:** The actual information stored in the database, such as text, numbers, images, or files.
- **Schema:** The structural blueprint that defines how data is organized—tables, fields, data types, and relationships.
- **DBMS:** The software that manages database operations like storage, retrieval, and security (e.g., MySQL, Oracle).
- **Queries:** Instructions (usually SQL) used to retrieve or manipulate data within the database.
- **Users:** People or systems that interact with the database, each with specific roles and access permissions.

Types of Databases

Databases can be classified into two primary types **Relational (SQL)** and **NoSQL Databases**. NoSQL is then further divided into four types: Document-oriented, Key-Value, Wide-Column, and Graph databases.



1. Relational Databases (RDBMS)

Relational databases organize data into tables made up of rows (records) and columns (fields). They use schemas (blueprints) to define how data is structured and how different tables relate to each other.

- Strict schema-based structure.
- Primary Keys (unique IDs) and Foreign Keys (relationships between tables).
- Strong ACID compliance (Atomicity, Consistency, Isolation, Durability).
- Ideal for structured data.



Examples: MySQL, PostgreSQL, Oracle, Microsoft SQL Server.

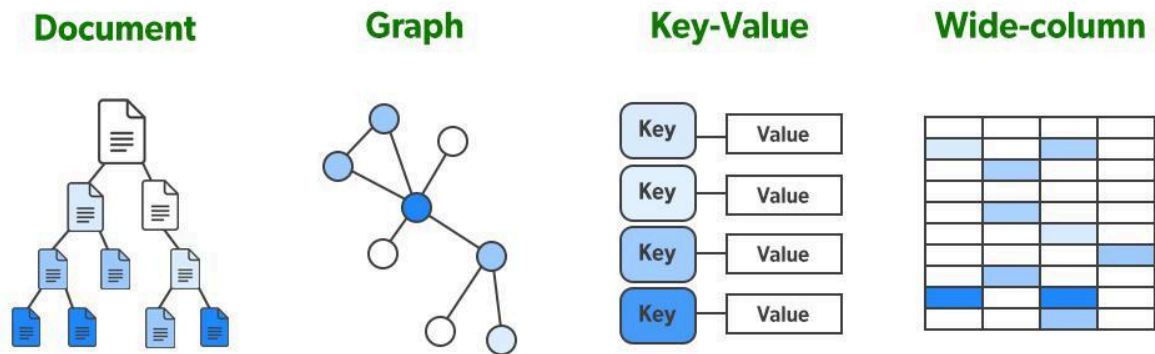
2. NoSQL Databases

"NoSQL" stands for "Not Only SQL". These databases are designed to handle unstructured or semi-structured data, such as text, images, videos or sensor data. They don't rely on the traditional table format.

- Flexible data models (no fixed schema).

- Scales horizontally for high-volume data.
- Often optimized for specific use cases like graphs or time-series data.

Main Sub-Types of NoSQL Databases:



1. **Document Databases** – Store data as JSON-like documents. (Example: MongoDB)
2. **Key-Value Stores** – Store simple key–value pairs for fast lookups. (Example: Redis)
3. **Columnar Databases** – Store data by columns for analytics. (Example: Apache Cassandra)
4. **Graph Databases** – Store nodes & relationships for connected data. (Example: Neo4j)

ACID Properties

ACID stands for Atomicity, Consistency, Isolation, and Durability—four essential principles that ensure your database transactions are reliable, accurate, and secure.

- **Atomicity:** Ensures transactions complete fully or not at all.
- **Consistency:** Ensures the database moves from one valid state to another.
- **Isolation:** Ensures that multiple transactions can happen at the same time without affecting each other.
- **Durability:** Saves changes permanently after completion.

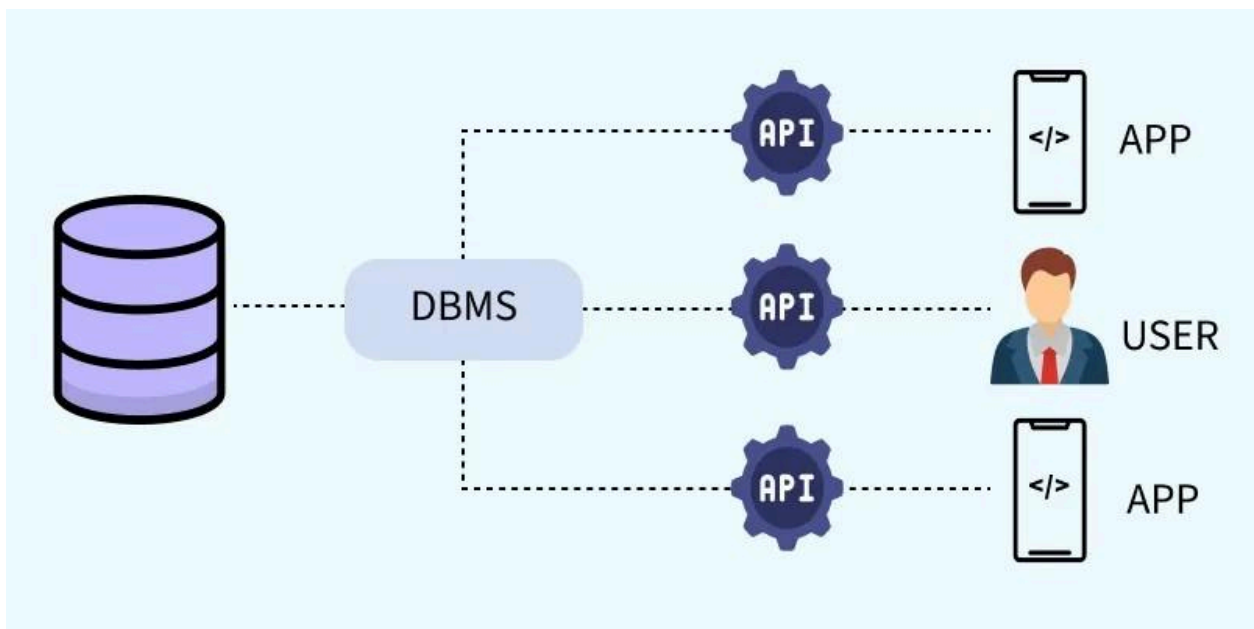
Real-World Applications of Databases

Databases are essential part of our life. There are several everyday activities that involve our interaction with databases.

- **Banking:** Stores transactions and account details.
- **Transportation:** Manages bookings and schedules.
- **Education:** Tracks student records and grades.
- **Retail:** Handles inventory and customer orders.
- **Social Media:** Stores user data, messages, and media.
- **Multimedia:** Manages images, audio, and video.
- **Business & Data Science:** Analyzes trends and supports predictions.

DBMS is a software system that manages, stores, and retrieves data efficiently in a structured format.

- It allows users to create, update, and query databases efficiently.
- Ensures data integrity, consistency, and security across multiple users and applications.
- Reduces data redundancy and inconsistency through centralized control.
- Supports concurrent data access, transaction management, and automatic backups



DBMS acts as a bridge between a central database and multiple clients-including apps and users. It uses **APIs** to handle data requests, enabling apps and users to interact with the database securely and efficiently without directly accessing the data.

Problems with Traditional File-Based Systems

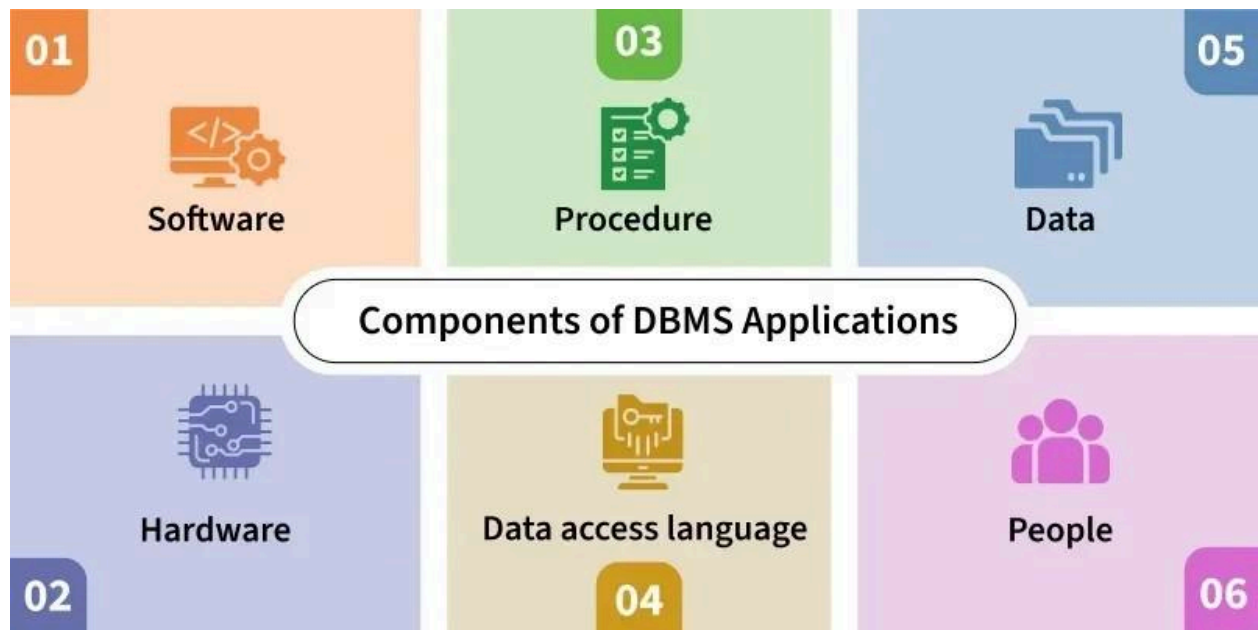
Before the introduction of modern DBMS, data was managed using basic file systems on hard drives. While this approach allowed users to store, retrieve and update files as needed, it came with numerous challenges

- **Data Redundancy:** Duplicate entries across files
- **Inconsistency:** Conflicting or outdated information
- **Difficult Access:** Manual file search required
- **Poor Security:** No control over data access
- **Single-User Access:** No support for collaboration
- **No Backup/Recovery:** Data loss was often permanent

A university file-based system storing data in separate files (e.g., Academics, Results, Hostels) often faced these problems.

Components of DBMS Applications

Any DBMS based applications is made up of six key components that work together to handle data effectively.



Components of DBMS Applications

1. Hardware

- Physical devices like servers, disks, input-output devices (keyboard, monitor, printer).
- Stores and processes data; interfaces between real-world inputs and digital systems.
- Examples: Personal computer hard disk, RAM, network devices used for DBMS operations.

2. Software

- Actual DBMS software like MySQL, Oracle, PostgreSQL.
- Includes the database engine, OS, network software, and application tools.
- Translates database access languages into operations.

3. Data

- Raw facts stored in structured or unstructured formats.
- **Operational Data:** Actual user data (e.g., name, age).
- **Metadata:** Data about data (e.g., storage time, size, data type).
- Core reason DBMS exists—to manage and store data efficiently.

4. Procedures

- Instructions and rules for using DBMS effectively.
- Covers setup, login/logout, data validation, backup, access control, and report generation.
- Helps ensure consistent and secure use of the system.

5. Database Access Language

- Used to interact with the database (create, read, update, delete data).
- Examples: SQL, MyAccess, Oracle PL/SQL.
- **DDL (Data Definition Language)** – CREATE, ALTER, DROP
- **DML (Data Manipulation Language)** – INSERT, UPDATE, DELETE

6. People

- Users interacting with DBMS at different levels:
- **Database Administrators (DBA)** – Manage security, performance, user access.

- **Developers** – Build applications using the database.
- **End Users** – Use applications to access the database (e.g., students, employees).

Types of DBMS

There are several types of Database Management Systems (DBMS), each tailored to different data structures, scalability requirements and application needs. The most common types are as follows:

1. Relational Database Management System (RDBMS)

- It organizes data into tables (relations) composed of rows and columns.
- Uses primary keys to uniquely identify rows and foreign keys to establish relationships between tables.
- Queries are written in **SQL (Structured Query Language)**, which allows for efficient data manipulation and retrieval.

Examples: MySQL oracle, Microsoft SQL Server and Postgre SQL.

2. NoSQL DBMS

- They are designed to handle large-scale data and provide high performance for scenarios where relational models might be restrictive.
- They store data in various non-relational formats, such as key-value pairs, documents, graphs or columns.

- These flexible data models enable rapid scaling and are well-suited for unstructured or semi-structured data.

Examples: MongoDB, Cassandra, DynamoDB and Redis.

3. Object-Oriented DBMS (OODBMS)

- It integrates object-oriented programming concepts into the database environment, allowing data to be stored as objects.
- Supports complex data types and relationships, making it ideal for applications requiring advanced data modeling and real-world simulations.

Examples: ObjectDB, db4o.

4. Hierarchical Database

- Organizes data in a tree-like structure, where each record (node) has a single parent and have multiple children.
- This model is similar to a file system with folders and subfolders.
- It is efficient for storing data with a clear hierarchy, such as organizational charts or file directories.
- Navigation is fast and predictable due to the fixed structure.
- It lacks flexibility and difficult to restructure or handle complex many-to-many relationships.

Example: IBM Information Management System (IMS).

5. Network Database

- It uses a graph-like model to allow more complex relationships between entities.
- Unlike the hierarchical model, it permits each child to have multiple parents, enabling many-to-many relationships.
- Data is represented using records and sets, where sets define the relationships.
- It is more flexible than the hierarchical model and better suited for applications with complex data linkages.

Example: Integrated Data Store (IDS), TurboIMAGE.

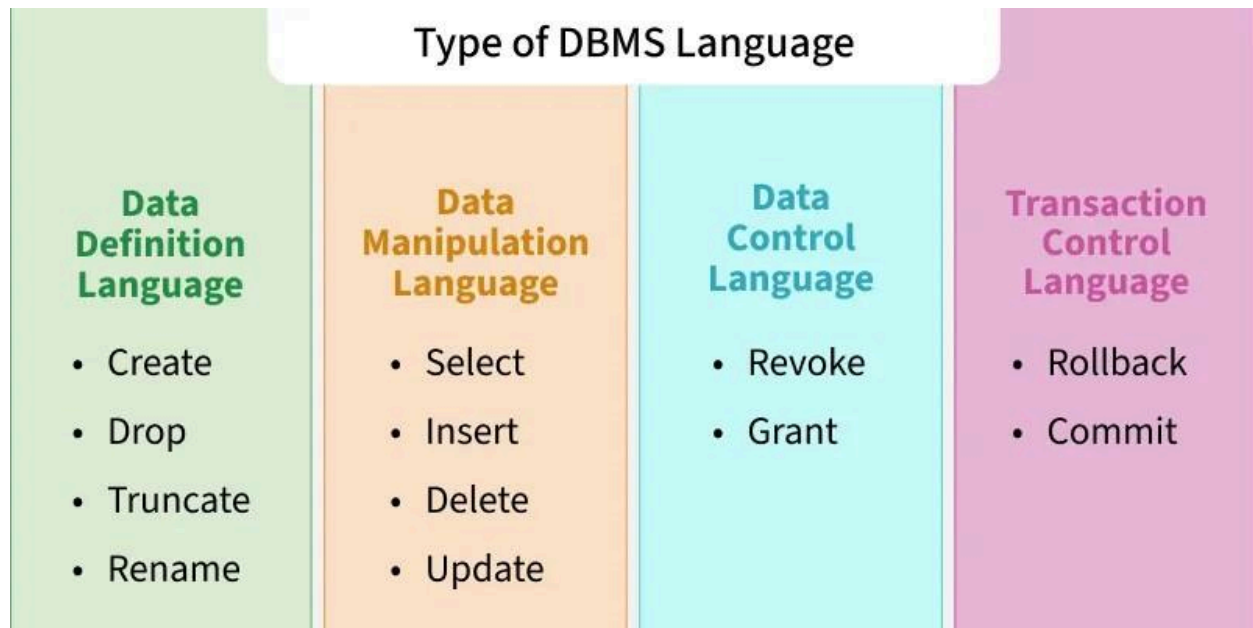
6. Cloud-Based Database

- They are hosted on cloud computing platforms like AWS, Azure or Google Cloud.
- They offer on-demand scalability, high availability, automatic backups and remote accessibility.
- These databases can be relational (SQL) or non-relational (NoSQL) and are maintained by cloud service providers, reducing administrative overhead.
- They support modern application requirements, including distributed access and real-time analytics.

Example: Amazon RDS (for SQL), MongoDB Atlas (for NoSQL), Google BigQuery.

Database Languages

Database languages are specialized sets of commands and instructions used to define, manipulate and control data within a database. Each language type plays a distinct role in database management, ensuring efficient storage, retrieval and security of data. The primary database languages include:



1. Data Definition Language (DDL)

DDL is the short name for Data Definition Language, which deals with database schemas and descriptions, of how the data should reside in the database.

- **CREATE:** to create a database and its objects like (table, index, views, store procedure, function and triggers)
- **ALTER:** alters the structure of the existing database
- **DROP:** delete objects from the database
- **TRUNCATE:** remove all records from a table, including all spaces allocated for the records are removed
- **COMMENT:** add comments to the data dictionary

- **RENAME:** rename an object

2. Data Manipulation Language (DML)

DML focuses on manipulating the data stored in the database, enabling users to retrieve, add, update and delete data.

- **SELECT:** retrieve data from a database
- **INSERT:** insert data into a table
- **UPDATE:** updates existing data within a table
- **DELETE:** Delete all records from a database table
- **MERGE:** UPSERT operation (insert or update)
- **CALL:** call a PL/SQL or Java subprogram
- **EXPLAIN PLAN:** interpretation of the data access path
- **LOCK TABLE:** concurrency Control

3. Data Control Language (DCL)

DCL commands manage access permissions, ensuring data security by controlling who can perform certain actions on the database.

- **GRANT:** Provides specific privileges to a user (e.g., SELECT, INSERT).
- **REVOKE:** Removes previously granted permissions from a user.

4. Transaction Control Language (TCL)

TCL commands oversee transactional data to maintain consistency, reliability and atomicity.

- **ROLLBACK**: Undoes changes made during a transaction.
- **COMMIT**: Saves all changes made during a transaction.
- **SAVEPOINT**: Sets a point within a transaction to which one can later roll back.

5. Data Query Language (DQL)

DQL is a subset of DML, specifically focused on data retrieval.

- **SELECT**: The primary DQL command, used to query data from the database without altering its structure or contents.

Applications of DBMS



- **Banking:** Manages accounts and transactions.
- **E-commerce:** Tracks products, orders, and customers.
- **Healthcare:** Stores patient records and diagnoses.
- **Education:** Handles student grades and schedules.
- **Social Media:** Manages user profiles and interactions.
- **Data Science:** Supports analytics and predictions.

DBMS Architecture 1-level, 2-Level, 3-Level

A DBMS architecture defines how users interact with the database to read, write, or update information. A well-designed architecture and schema (a blueprint detailing tables, fields and relationships) ensure data consistency, improve performance and keep data secure.

Types of DBMS Architecture

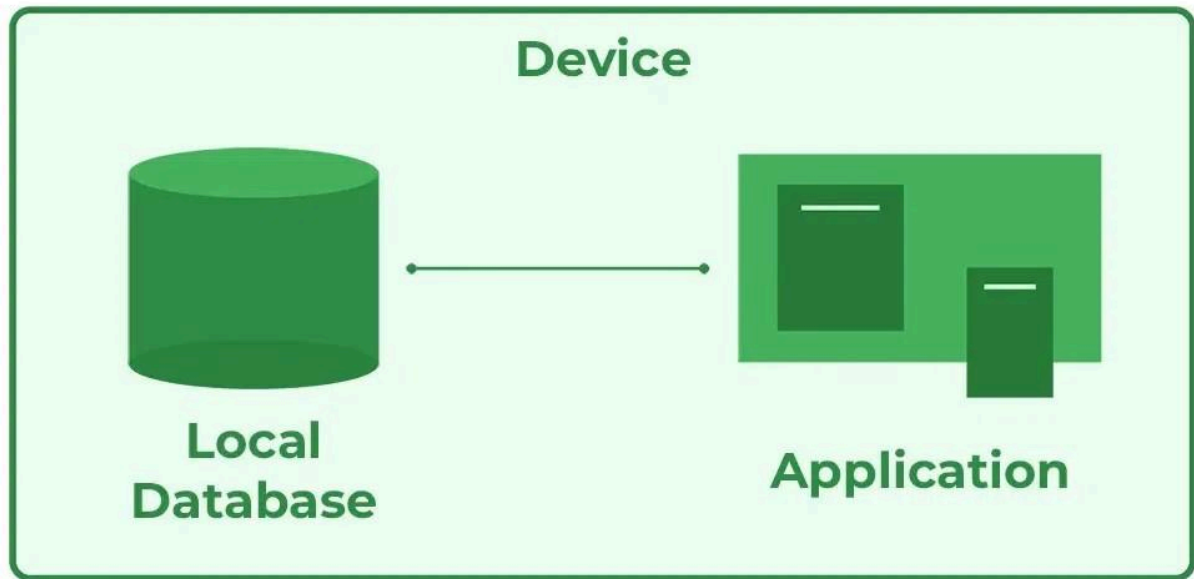
There are several types of DBMS Architecture that we use according to the usage requirements.

- 1-Tier Architecture
- 2-Tier Architecture
- 3-Tier Architecture

1-Tier Architecture

In 1-Tier Architecture, the user works directly with the database on the same system. This means the client, server and database are all in one application. The user can

open the application, interact with the data and perform tasks without needing a separate server or network connection.



DBMS 1-Tier Architecture

- A common example is Microsoft Excel. Everything from the user interface to the logic and data storage happens on the same device. The user enters data, performs calculations and saves files directly on their computer.
- This setup is simple and easy to use, making it ideal for personal or standalone applications. It does not require a network or complex setup, which is why it's often used in small-scale or individual use cases.
- This architecture is simple and works well for personal, standalone applications where no external server or network connection is needed.

Advantages of 1-Tier Architecture

Below mentioned are the advantages of 1-Tier Architecture.

- **Simple Architecture:** 1-Tier Architecture is the most simple architecture to set up, as only a single machine is required to maintain it.
- **Cost-Effective:** No additional hardware is required for implementing 1-Tier Architecture, which makes it cost-effective.
- **Easy to Implement:** 1-Tier Architecture can be easily deployed and hence it is mostly used in small projects.

Disadvantages of 1-Tier Architecture

- **Limited to Single User:** Only one person can use the application at a time. It's not designed for multiple users or teamwork.
- **Poor Security:** Since everything is on the same machine, if someone gets access to the system, they can access both the data and the application easily.
- **No Centralized Control:** Data is stored locally, so there's no central database. This makes it hard to manage or back up data across multiple devices.
- **Hard to Share Data:** Sharing data between users is difficult because everything is stored on one computer.

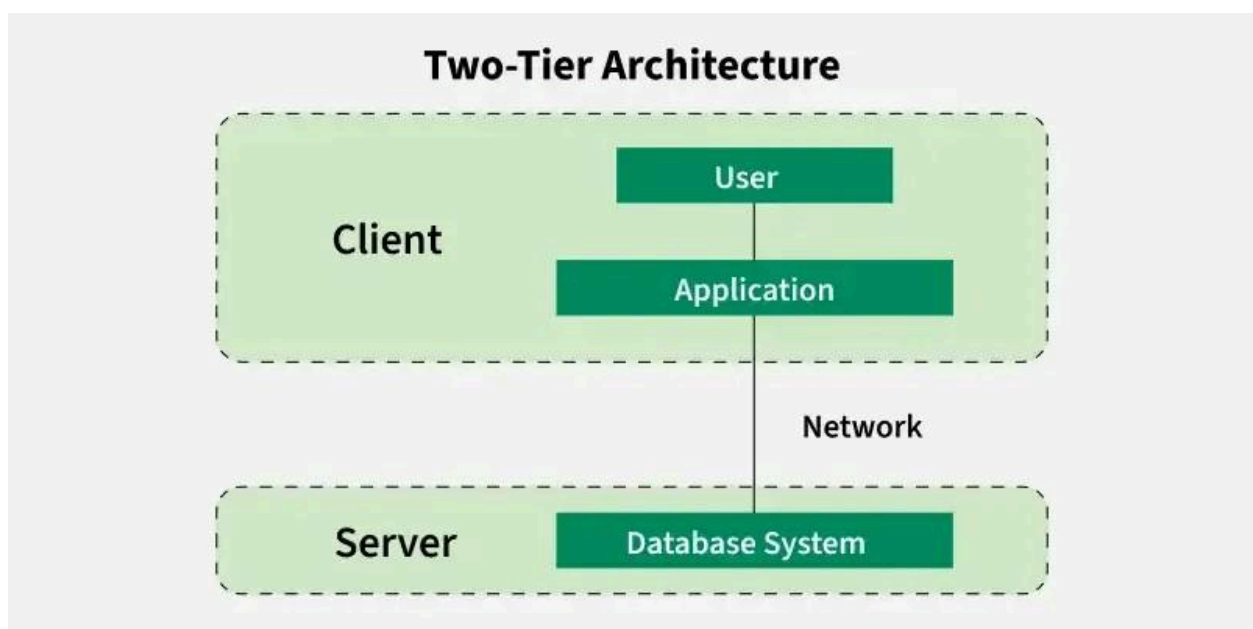
2-Tier Architecture

The 2-tier architecture is similar to a basic client-server model.

The Client-Server Model is a distributed architecture where clients request services and servers provide them. Clients send requests to servers, which process them and return the results. Clients don't share resources among themselves but depend on the

server. Common examples include email systems and the World Wide Web, where clients (like browsers or email apps) interact with servers to access content or send data.

The application at the client end directly communicates with the database on the server side. APIs like ODBC and JDBC are used for this interaction. The server side is responsible for providing query processing and transaction management functionalities.



DBMS 2-Tier Architecture

- On the client side, the user interfaces and application programs are run. The application on the client side establishes a connection with the server side to communicate with the DBMS. For Example: A Library Management System used in schools or small organizations is a classic example of two-tier architecture.
- **Client Layer (Tier 1):** This is the user interface that library staff or users interact with. For example they might use a desktop application to search for books, issue them, or check due dates.

- **Database Layer (Tier 2):** The database server stores all the library records such as book details, user information and transaction logs.
- The client layer sends a request (like searching for a book) to the database layer which processes it and sends back the result. This separation allows the client to focus on the user interface, while the server handles data storage and retrieval.

Advantages of 2-Tier Architecture

- **Easy to Access:** 2-Tier Architecture makes easy access to the database, which makes fast retrieval.
- **Scalable:** We can scale the database easily, by adding clients or upgrading hardware.
- **Low Cost:** 2-Tier Architecture is cheaper than 3-Tier Architecture and [Multi-Tier Architecture](#).
- **Easy Deployment:** 2-Tier Architecture is easier to deploy than 3-Tier Architecture.
- **Simple:** 2-Tier Architecture is easily understandable as well as simple because of only two components.

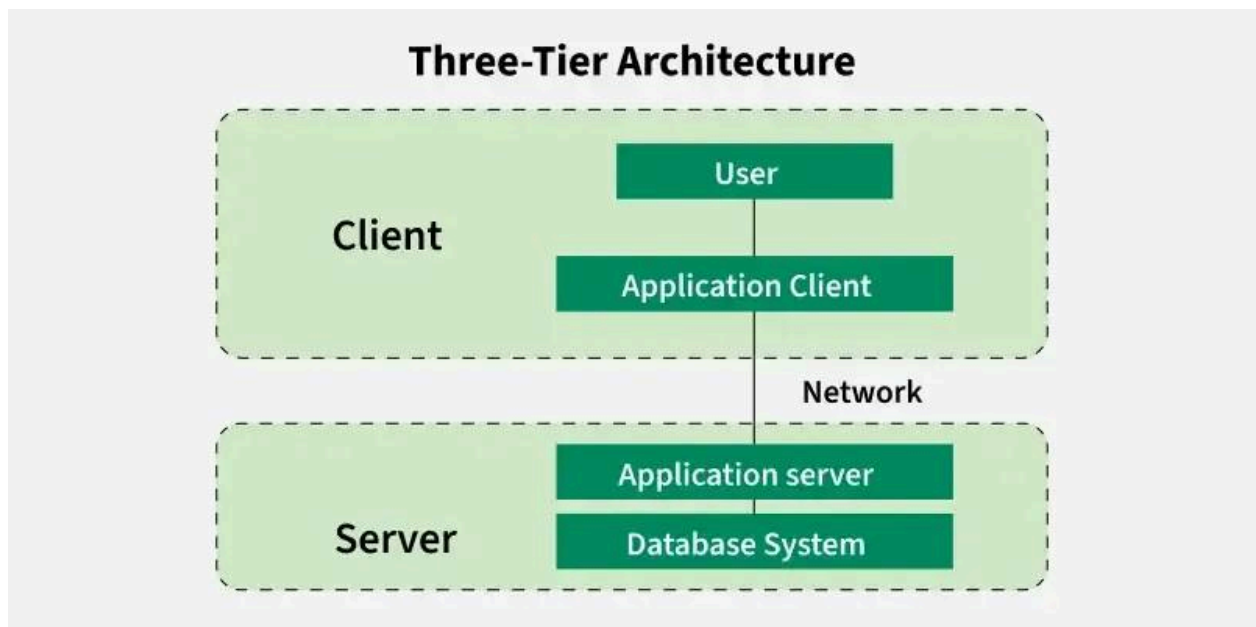
Disadvantages of 2-Tier Architecture

- **Limited Scalability:** As the number of users increases, the system performance can slow down because the server gets overloaded with too many requests.

- **Security Issues:** Clients connect directly to the database, which can make the system more vulnerable to attacks or data leaks.
- **Tight Coupling:** The client and the server are closely linked. If the database changes, the client application often needs to be updated too.
- **Difficult Maintenance:** Managing updates, fixing bugs, or adding features becomes harder when the number of users or systems increases.

3-Tier Architecture

In 3-Tier Architecture, there is another layer between the client and the server. The client does not directly communicate with the server. Instead, it interacts with an application server which further communicates with the database system and then the query processing and transaction management takes place. This intermediate layer acts as a medium for the exchange of partially processed data between the server and the client. This type of architecture is used in the case of large web applications.



DBMS 3-Tier Architecture

Example: E-commerce Store

- **User:** You visit an online store, search for a product and add it to your cart.
- **Processing:** The system checks if the product is in stock, calculates the total price and applies any discounts.
- **Database:** The product details, your cart and order history are stored in the database for future reference.

Advantages of 3-Tier Architecture

- **Enhanced scalability:** Scalability is enhanced due to the distributed deployment of application servers. Now, individual connections need not be made between the client and server.
- **Data Integrity:** 3-Tier Architecture maintains Data Integrity. Since there is a middle layer between the client and the server, data corruption can be avoided/removed.
- **Security:** 3-Tier Architecture Improves Security. This type of model prevents direct interaction of the client with the server thereby reducing access to unauthorized data.

Disadvantages of 3-Tier Architecture

- **More Complex:** 3-Tier Architecture is more complex in comparison to 2-Tier Architecture. Communication Points are also doubled in 3-Tier Architecture.
- **Difficult to Interact:** It becomes difficult for this sort of interaction to take place due to the presence of middle layers.

- **Slower Response Time:** Since the request passes through an extra layer (application server), it may take more time to get a response compared to 2-Tier systems.
- **Higher Cost:** Setting up and maintaining three separate layers (client, server and database) requires more hardware, software and skilled people. This makes it more expensive.

Data Abstraction and Data Independence

Database systems comprise complex data structures. In order to make the system efficient in terms of retrieval of data, and reduce complexity in terms of usability of users, developers use abstraction i.e. hide irrelevant details from the users. This approach simplifies database design.

Level of Abstraction in a DBMS

There are mainly 3 levels of data abstraction:

- Physical or Internal Level
- Logical or Conceptual Level
- View or External Level

Physical or Internal Level

This is the lowest level of data abstraction. It tells us how the data is actually stored in memory. Access methods like sequential or random access and file organization methods like B+ trees and hashing are used for the same. Usability, size of memory, and the number of times the records are factors that we need to know while designing the database.

Suppose we need to store the details of an employee. Blocks of storage and the amount of memory used for these purposes are kept hidden from the user.

Logical or Conceptual Level

This level comprises the information that is actually stored in the database in the form of tables. It also stores the relationship among the data entities in relatively simple structures. At this level, the information available to the user at the view level is unknown.

We can store the various attributes of an employee and relationships, e.g. with the manager can also be stored.

The logical level thus describes the entire database in terms of a small number of relatively simple structures. Although implementation of the simple structures at the logical level may involve complex physical-level structures, the user of the logical level does not need to be aware of this complexity. This is referred to as physical data independence. Database administrators, who must decide what information to keep in the database, use the logical level of abstraction.

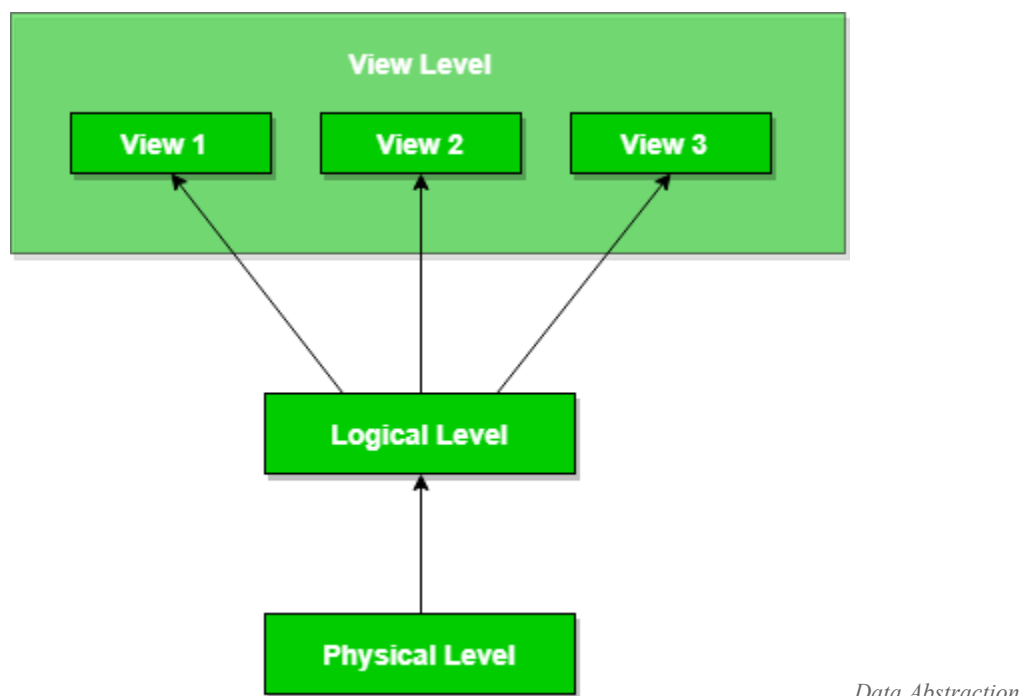
View or External Level

This is the highest level of abstraction. Only a part of the actual database is viewed by the users. This level exists to ease the accessibility of the database by an individual user. Users view data in the form of rows and columns. Tables and relations are used to store data. Multiple views of the same database may exist. Users can just view the data and interact with the database, storage and implementation details are hidden from them. Even though the logical level uses simpler structures, complexity remains because of the variety of information stored in a large database. Many users of the database system do not need all this information; instead, they need to access only a

part of the database. The view level of abstraction exists to simplify their interaction with the system

Example: In case of storing customer data,

- **Physical level** - it will contains block of storages (bytes,GB,TB,etc)
- **Logical level** - it will contain the fields and the attributes of data.
- **View level** - it works with CLI or GUI access of database



The main purpose of data abstraction is to achieve data independence in order to save the time and cost required when the database is modified or altered.

Data Independence

Data Independence is mainly defined as a property of DBMS that helps you to change the database schema at one level of a system without requiring to change the schema at the next level. it helps to keep the data separated from all program that

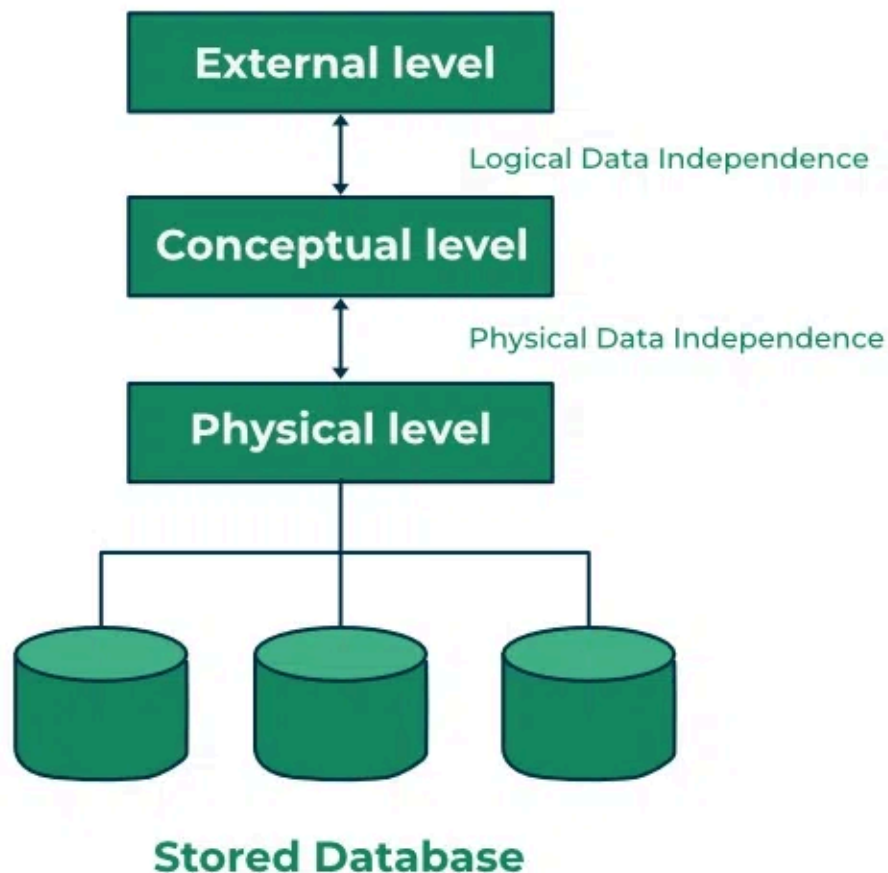
makes use of it.

We have namely two levels of data independence arising from these levels of abstraction:

Physical level data independence

Logical level data independence

Data Independence in DBMS



Physical Level Data Independence

It refers to the characteristic of being able to modify the physical schema without any alterations to the conceptual or logical schema, done for optimization purposes, e.g., the Conceptual structure of the database would not be affected by any change in storage size of the database system server. Changing from sequential to random access files is one such example. These alterations or modifications to the physical structure may include:

- Utilizing new storage devices.
- Modifying data structures used for storage.
- Altering indexes or using alternative file organization techniques etc.

Logical Level Data Independence

It refers characteristic of being able to modify the logical schema without affecting the external schema or application program. The user view of the data would not be affected by any changes to the conceptual view of the data. These changes may include insertion or deletion of attributes, altering table structures entities or relationships to the logical schema, etc.

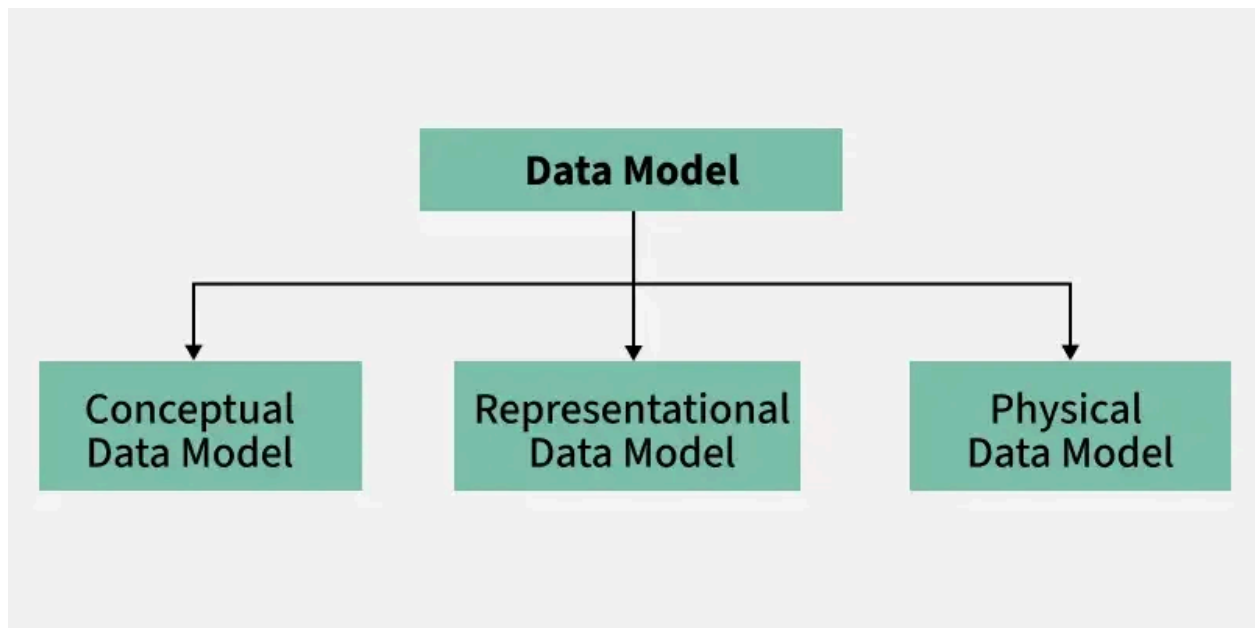
Data Models in DBMS

A Data Model in Database Management System (DBMS) is the concept of tools that are developed to summarize the description of the database. Data Models provide us with a transparent picture of data which helps us in creating an actual database. It shows us from the design of the data to its proper implementation of data.

Types of Relational Models

1. Conceptual Data Model
2. Representational Data Model
3. Physical Data Model

It is basically classified into 3 types:-



1. Conceptual Data Model

The conceptual data model describes the database at a very high level and is useful to understand the needs or requirements of the database. It is this model, that is used in the requirement-gathering process i.e. before the Database Designers start making a particular database. One such popular model is the entity/relationship model (ER model). The E/R model specializes in entities, relationships, and even attributes that are used by database designers. In terms of this concept, a discussion can be made even with non-computer science(non-technical) users and stakeholders, and their requirements can be understood.

Entity-Relationship Model(ER Model): It is a high-level data model which is used to define the data and the relationships between them. It is basically a conceptual design of any database which is easy to design the view of data.

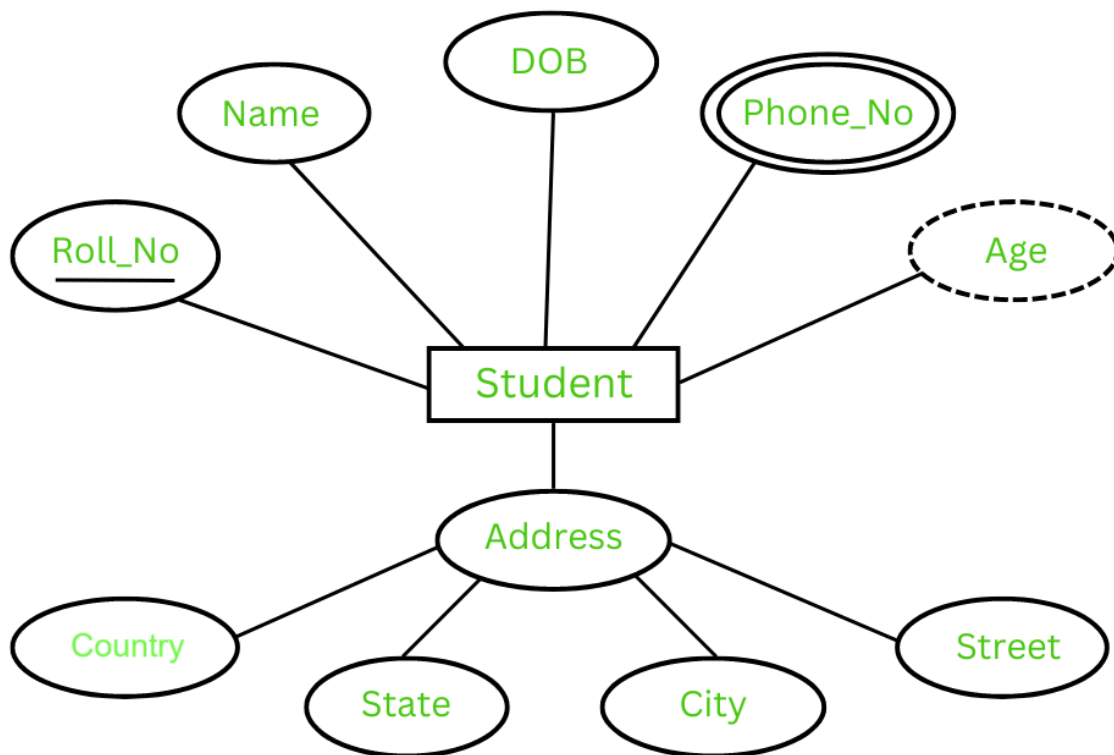
Components of ER Model:

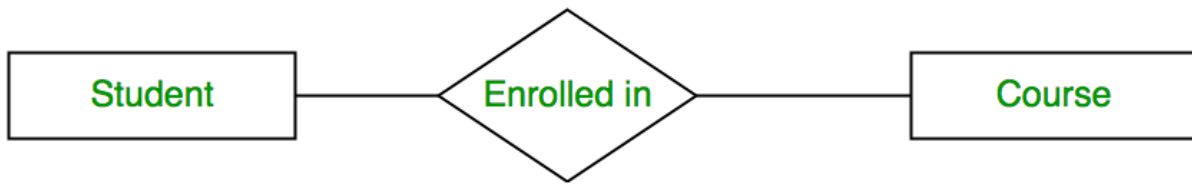
1. **Entity:** An entity is referred to as a real-world object. It can be a name, place, object, class, etc. These are represented by a rectangle in an ER Diagram.
2. **Attributes:** An attribute can be defined as the description of the entity. These are represented by Ellipse in an ER Diagram. It can be Age, Roll Number, or Marks for a Student.
3. **Relationship:** Relationships are used to define relations among different entities. Diamonds and Rhombus are used to show Relationships.

Characteristics of a conceptual data model

- Offers Organization-wide coverage of the business concepts.
- This type of Data Models are designed and developed for a business audience.
- The conceptual model is developed independently of hardware specifications like data storage capacity, location or software specifications like DBMS vendor and technology. The focus is to represent data as a user will see it in the “real world.”

Conceptual data models known as Domain models create a common vocabulary for all stakeholders by establishing basic concepts and scope

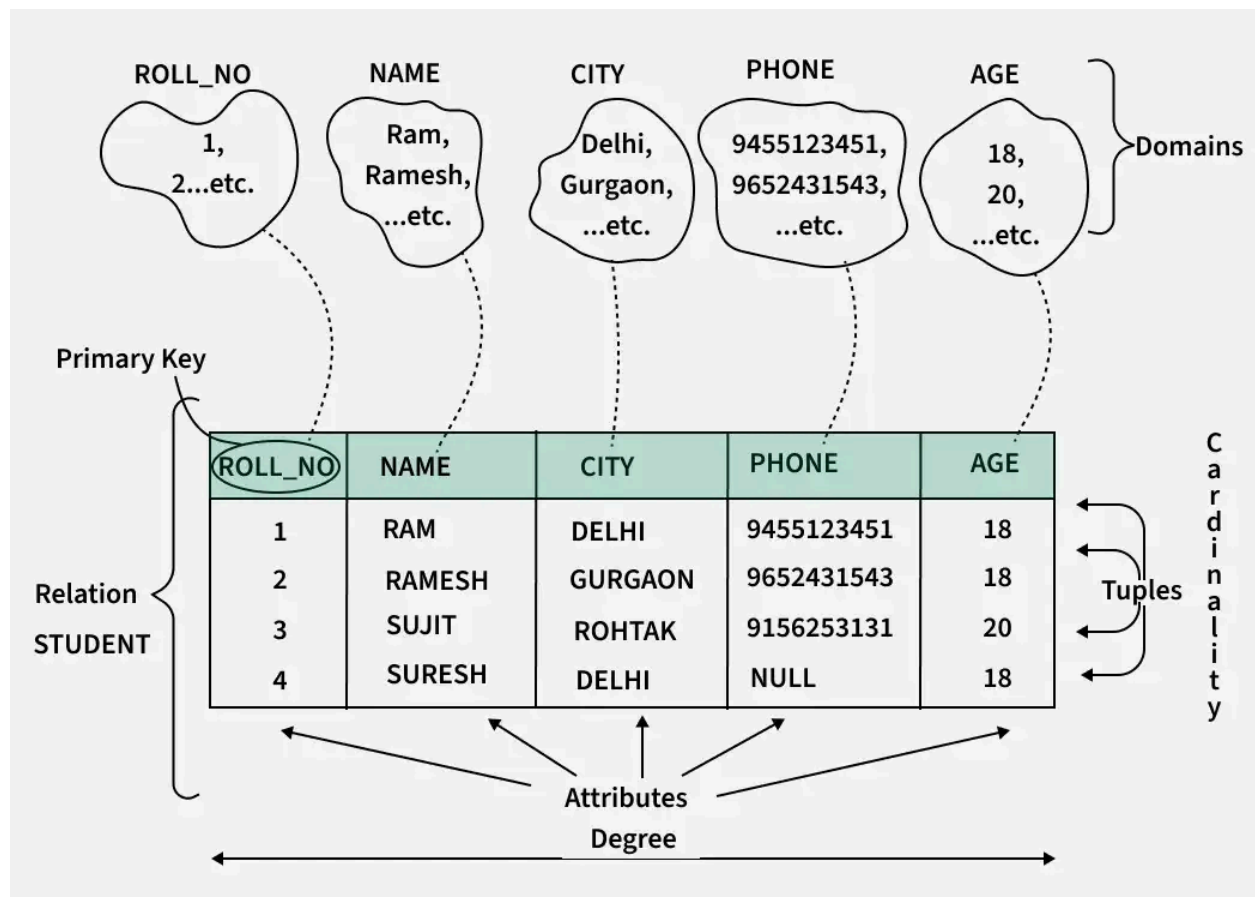




2. Representational Data Model

This type of data model is used to represent only the logical part of the database and does not represent the physical structure of the database. The representational data model allows us to focus primarily, on the design part of the database. A popular representational model is a Relational model. The relational Model consists of Relational Algebra and Relational Calculus. In the Relational Model, we basically use tables to represent our data and the relationships between them. It is a theoretical concept whose practical implementation is done in Physical Data Model.

The advantage of using a Representational data model is to provide a foundation to form the base for the Physical model.



Characteristics of Representational Data Model

- Represents the logical structure of the database.
- Relational models like Relational Algebra and Relational Calculus are commonly used.
- Uses tables to represent data and relationships.
- Provides a foundation for building the physical data model.

3. Physical Data Model

The physical Data Model is used to practically implement Relational Data Model. Ultimately, all data in a database is stored physically on a secondary storage device such as discs and tapes. This is stored in the form of files, records, and certain other data structures. It has all the information on the format in which the files are present and the structure of the databases, the presence of external data structures, and their relation to each other. Here, we basically save tables in memory so they can be accessed efficiently. In order to come up with a good physical model, we have to work on the relational model in a better way. [Structured Query Language \(SQL\)](#) is used to practically implement Relational Algebra.

This Data Model describes HOW the system will be implemented using a specific DBMS system. This model is typically created by DBA and developers. The purpose is actual implementation of the database.

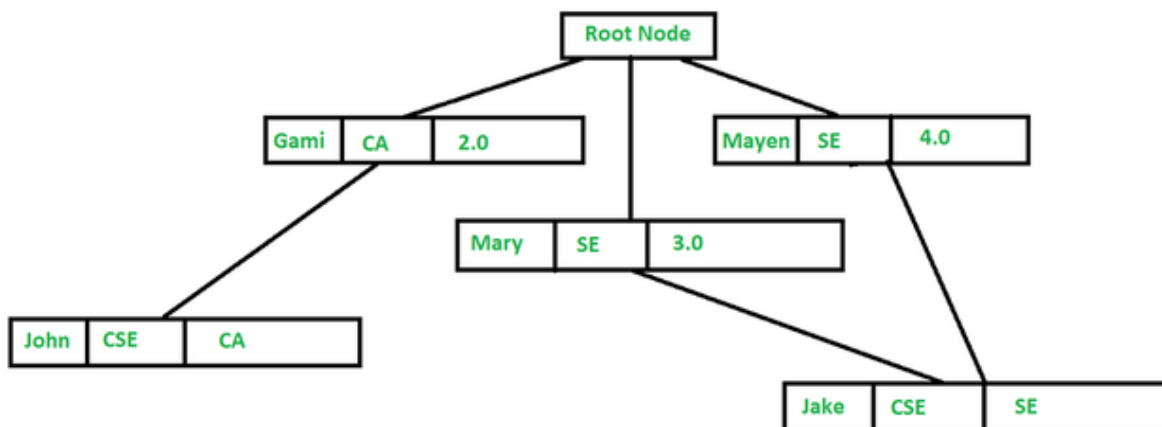
Characteristics of a physical data model:

- The physical data model describes data need for a single project or application though it maybe integrated with other physical data models based on project scope.
- Data Model contains relationships between tables that which addresses cardinality and nullability of the relationships.
- Developed for a specific version of a DBMS, location, data storage or technology to be used in the project.
- Columns should have exact datatypes, lengths assigned and default values.
- Primary and Foreign keys, views, indexes, access profiles, and authorizations, etc. are defined

Some Other Data Models

1. Hierarchical Model

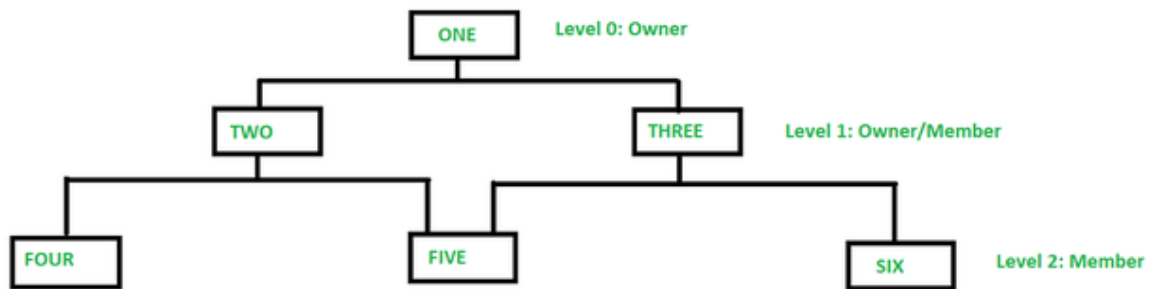
The [hierarchical Model](#) is one of the oldest models in the data model which was developed by IBM, in the 1950s. In a hierarchical model, data are viewed as a collection of tables, or we can say segments that form a hierarchical relation. In this, the data is organized into a tree-like structure where each record consists of one parent record and many children. Even if the segments are connected as a chain-like structure by logical associations, then the instant structure can be a fan structure with multiple branches. We call the illogical associations as directional associations.



2. Network Model

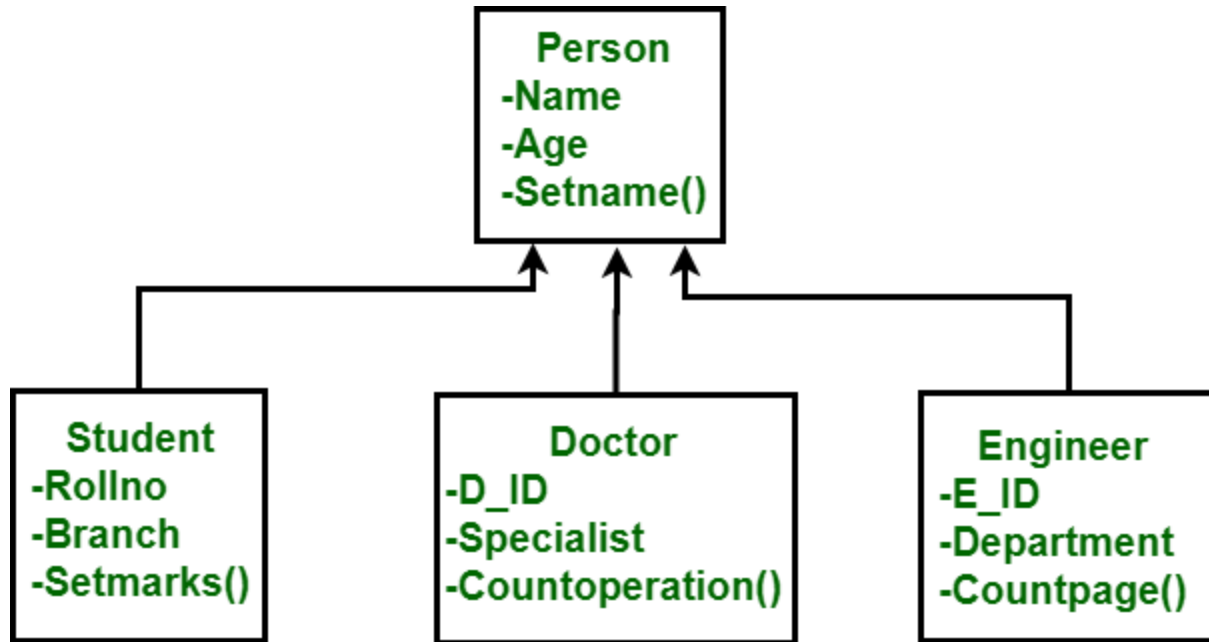
The [Network Model](#) was formalized by the Database Task group in the 1960s. This model is the generalization of the hierarchical model. This model can consist of multiple parent segments and these segments are grouped as levels but there exists a

logical association between the segments belonging to any level. Mostly, there exists a many-to-many logical association between any of the two segments.



3. Object-Oriented Data Model

In the [Object-Oriented Data Model](#), data and their relationships are contained in a single structure which is referred to as an object in this data model. In this, real-world problems are represented as objects with different attributes. All objects have multiple relationships between them. Basically, it is a combination of Object Oriented programming and a Relational Database Model.



4. Float Data Model

The float data model basically consists of a two-dimensional array of data models that do not contain any duplicate elements in the array. This data model has one drawback it cannot store a large amount of data that is the tables can not be of large size.

5. Context Data Model

The Context data model is simply a data model which consists of more than one data model. For example, the Context data model consists of ER Model, Object-Oriented Data Model, etc. This model allows users to do more than one thing which each individual data model can do.

6. Semi-Structured Data Model

Semi-Structured data models deal with the data in a flexible way. Some entities may have extra attributes and some entities may have some missing attributes. Basically, you can represent data here in a flexible way.

Advantages of Data Models

1. Data Models help us in representing data accurately.
2. It helps us in finding the missing data and also in minimizing Data Redundancy.
3. Data Model provides data security in a better way.
4. The data model should be detailed enough to be used for building the physical database.
5. The information in the data model can be used for defining the relationship between tables, primary and foreign keys, and stored procedures.

Disadvantages of Data Models

1. In the case of a vast database, sometimes it becomes difficult to understand the data model.
2. You must have the proper knowledge of [SQL](#) to use physical models.
3. Even smaller change made in structure require modification in the entire application.
4. There is no set data manipulation language in DBMS.
5. To develop Data model one should know physical data stored characteristics.

Conclusion

In conclusion, data modeling is a crucial process for designing databases that ensure consistency, quality, and accuracy in how data is stored and managed. It establishes a structured framework using conceptual, logical, and physical models to define entities, relationships, and implementation details. While data modeling supports better organization and ensures data integrity, it can be challenging due to the impact of structural changes on the entire application. Despite its drawbacks, data modeling is essential for creating efficient, reliable, and scalable database systems.